

Technical Interview Prep

Software Architect — API & Data Exchange Platform · .NET / Kubernetes / REST

ASKED confirmed asked in the interview · **LIKELY** natural follow-up on the same topic

1 · REST

Q What is REST? **ASKED**

REST (Representational State Transfer) is an **architectural style** for distributed systems, defined by a set of constraints rather than a protocol. A system is RESTful when it satisfies them:

- **Client-server** — separation of concerns; UI and data storage evolve independently.
- **Stateless** — each request carries all context needed; the server keeps no client session state between requests.
- **Cacheable** — responses declare whether they can be cached.
- **Uniform interface** — resources identified by URIs, manipulated through representations, self-descriptive messages, and HATEOAS.
- **Layered system** — client can't tell if it talks to the origin server or an intermediary (proxy, gateway).
- **Code on demand** (optional).

Resources are nouns (`/orders/42`), and HTTP verbs express the operation:

`GET` read, `POST` create, `PUT` replace, `PATCH` partial update, `DELETE` remove.

One-liner: "REST is a stateless, resource-oriented architectural style over HTTP, where resources have URIs and standard verbs define the operations — not a protocol or a framework."

Q REST vs. RESTful, and the Richardson Maturity Model **LIKELY**

Many APIs called "REST" are really RPC over HTTP. The **Richardson Maturity Model** grades how RESTful an API actually is:

- **Level 0** — single endpoint, HTTP as a tunnel (plain RPC).
- **Level 1** — separate resources, but verbs still in the URL.

- **Level 2** — proper HTTP verbs + status codes (where most "REST" APIs sit).
- **Level 3** — HATEOAS: responses include links to related actions/resources.

Q Idempotency & safe methods LIKELY

Safe = no state change (`GET` , `HEAD` , `OPTIONS`). **Idempotent** = same result whether called once or N times.

Method	Safe	Idempotent
GET	Yes	Yes
PUT	No	Yes
DELETE	No	Yes
POST	No	No
PATCH	No	Not necessarily

Practical angle: to make `POST` retry-safe under network failure, use an **idempotency key** so the server dedupes repeated submissions.

2 · API Versioning

Q How can a REST API be versioned? ASKED

Four common strategies, roughly in order of popularity:

Strategy	Example	Trade-off
URI path	<code>/v2/orders</code>	Most visible, easy to route/cache; "impure" (version in resource identifier)
Query param	<code>/orders?v=2</code>	Easy default; can clutter caching
Custom header	<code>X-API-Version: 2</code>	Clean URLs; less discoverable, harder to test in a browser
	<code>Accept: application/vnd.company.v2+json</code>	Most "RESTful"; most complex, least intuitive

Media type (content negotiation)		
--	--	--

What to emphasize: the goal of versioning is to evolve the contract *without breaking existing consumers*. State a preference (URI versioning is the pragmatic default for public APIs; header/media-type for stricter REST purity) and justify it — interviewers care about the reasoning, not the "right" answer.

Q How do you keep versions maintainable? LIKELY

- Version the **contract**, not every internal change. Additive changes shouldn't need a new version.
- Prefer **expand/contract (parallel change)**: add the new field/endpoint, migrate consumers, then remove the old one — rather than a hard cutover.
- Publish a **deprecation policy** (sunset dates, Deprecation / Sunset headers) so consumers get warning.
- Semantic versioning intent: **major** = breaking, **minor** = additive/backward-compatible, **patch** = fixes.

3 · Breaking Changes

Q What is a breaking change, and how does it relate to versioning? ASKED

A **breaking change** is any change to the contract that forces existing consumers to change their code to keep working. It's the trigger for a new **major** version.

Breaking (need a new major version)

- Removing or renaming a field, endpoint, or parameter.
- Making an optional request field required.
- Changing a field's type or semantic meaning.
- Tightening validation (rejecting input that used to be accepted).

- Changing response structure, error codes, or default behavior consumers relied on.
- Removing an enum value that clients might send.

Non-breaking (backward-compatible)

- Adding a new endpoint.
- Adding an *optional* request field.
- Adding a new field to a response **if** clients ignore unknowns (tolerant reader).
- Adding a new *optional* HTTP header.

Key principle — Robustness / Tolerant Reader: "Be conservative in what you send, liberal in what you accept." Consumers should ignore unknown fields so that additive changes stay non-breaking. Whether a change is breaking partly depends on how tolerant your clients are.

4 · Enum Evolution

Q Is adding a new value to an enum a breaking change? **ASKED**

It depends on direction — the honest, senior answer is "it depends, and here's on what."

Enum appears in...	Adding a value is...
Response (server → client)	Potentially breaking — a client with an exhaustive switch or strict deserializer may crash or mis-handle the unknown value.
Request (client → server)	Generally non-breaking — existing clients simply never send the new value; new clients opt in.

Why response-side is risky:

- Strict enum deserialization (e.g. failing on unknown names) throws at parse time.
- Exhaustive `switch` with no default silently skips the new case.
- Client-side validation may reject the payload outright.

How to make enums evolvable:

- Design clients as **tolerant readers**: always have a `default` / "unknown" branch.
- Document that enums are **open** (may gain values) vs. **closed** (fixed set) as part of the contract.
- In **.NET**: unmapped values deserialize to the underlying integer by default (a class-level issue if code assumes only named members), so guard with `Enum.IsDefined` or a fallback member.
- Consider a reserved `UNKNOWN = 0` member (Protobuf/gRPC convention) so unset/unrecognized maps to a safe default.

Crisp answer: "Adding an enum value is non-breaking in requests but potentially breaking in responses, because existing clients may not handle the unknown value. It's safe if clients are written as tolerant readers with a default case — so it's really a contract-design question."

5 · CQRS

Q What is CQRS, and what advantage does it give? ASKED

CQRS = Command Query Responsibility Segregation. It splits the model into two paths:

- **Commands** — change state, return no data (ideally). The write side.
- **Queries** — return data, never change state. The read side.

Instead of one model serving both, you have a **write model** and a **read model**, each optimized for its job.

Advantages

- **Independent optimization** — the write side enforces invariants and consistency; the read side is shaped for fast querying (denormalized, precomputed views).
- **Independent scaling** — reads usually vastly outnumber writes; scale them separately.
- **Different storage per side** — e.g. a normalized SQL write store and a denormalized/NoSQL or search-optimized read store.

- **Simpler models** — each side avoids the compromises of a shared do-everything model.
- **Security/clarity** — write concerns (validation, authorization) stay separate from read concerns.

Costs (mention these — shows balance)

- Added complexity; not justified for simple CRUD.
- If read and write stores are separate, you inherit **eventual consistency** — the read side lags the write side.
- More moving parts to keep in sync (often via events/projections).

Common misconceptions to correct:

- CQRS does **not** require Event Sourcing — they pair well but are independent.
- CQRS does **not** mandate two databases — you can apply it as just separate read/write models in one store.

Q CQRS vs. REST — are they alternatives? ASKED

They are not alternatives — they operate at different levels.

- **REST** is an *external* architectural style: how you expose resources and operations over an interface (usually HTTP).
- **CQRS** is an *internal* design pattern: how you structure command vs. query handling behind that interface.

A REST API can sit on top of a CQRS system: a **POST / PUT** maps to a command, a **GET** maps to a query. The client sees plain REST; CQRS is an implementation detail.

6 · Kubernetes & Containers

Q Difference between a pod and a container? ASKED

A **container** is a single isolated, runnable unit — app + dependencies packaged together (e.g. a Docker image running as a process).

A **pod** is Kubernetes' **smallest deployable unit** and wraps one or more containers that are scheduled together and share:

- **Network** — one IP per pod; containers reach each other over `localhost`.
- **Storage** — shared volumes mounted into the pod.
- **Lifecycle** — co-located on the same node, created/destroyed together.

Kubernetes never schedules a bare container — it always schedules pods. **Usually one container per pod**; multiple containers only when they're tightly coupled (e.g. a **sidecar** for logging, a proxy, or an init helper).

One-liner: "A container is a single packaged process; a pod is the Kubernetes wrapper around one or more containers that share a network namespace, storage, and lifecycle. Kubernetes schedules pods, not containers."

Q Related K8s objects worth knowing LIKELY

Object	Purpose
Deployment	Declarative desired state for pods; manages ReplicaSets, rolling updates, rollbacks.
ReplicaSet	Keeps N replicas of a pod running.
Service	Stable network endpoint + load balancing across pods (pods are ephemeral, IPs change).
Ingress	HTTP(S) routing from outside the cluster to Services.
ConfigMap / Secret	Externalized configuration and sensitive data.
Namespace	Logical partition of cluster resources.
StatefulSet	Pods with stable identity/storage (databases, etc.).

Q Container vs. VM LIKELY

Containers share the host kernel and isolate at the process level — lightweight, fast to start, less overhead. VMs virtualize hardware and run a full guest OS — stronger isolation, heavier. Docker packages the

container image; Kubernetes orchestrates many containers across a cluster.

7 · Architecture Principles (from the JD)

The role explicitly lists these — expect at least one to come up.

Q SOLID LIKELY

- **Single Responsibility** — a class has one reason to change.
- **Open/Closed** — open for extension, closed for modification.
- **Liskov Substitution** — subtypes must be usable wherever the base type is expected.
- **Interface Segregation** — many small, focused interfaces over one fat one.
- **Dependency Inversion** — depend on abstractions, not concretions.

Q Domain-Driven Design (DDD) LIKELY

- **Ubiquitous language** shared by devs and domain experts.
- **Bounded context** — an explicit boundary within which a model is consistent (ties directly to microservice boundaries).
- Building blocks: **Entity** (identity), **Value Object** (defined by attributes), **Aggregate** (consistency boundary with a root), **Repository**, **Domain Event**.

Q Conway's Law LIKELY

"Organizations design systems that mirror their own communication structure." Practical takeaway: system architecture and team structure shape each other — if you want a modular architecture (e.g. microservices), align team boundaries with service boundaries (the **Inverse Conway Maneuver**). The JD calls this out by name.

Q TDD LIKELY

- **Red → Green → Refactor**: write a failing test, make it pass minimally, then clean up.

- Benefits: design pressure toward testable units, living documentation, regression safety.
- Test pyramid: many unit tests, fewer integration, fewest end-to-end.

Q Microservices trade-offs LIKELY

- **For:** independent deploy/scale, team autonomy, tech heterogeneity, fault isolation.
- **Against:** distributed-system complexity, network latency, data consistency across services, operational overhead, harder debugging.
- Cross-service consistency: **Saga** pattern (choreography vs. orchestration); avoid distributed transactions.

8 · SQL & NoSQL

Q SQL vs. NoSQL — when which? LIKELY

	SQL (relational)	NoSQL
Schema	Fixed, enforced	Flexible / schemaless
Consistency	Strong (ACID)	Often eventual (BASE)
Scaling	Vertical (mostly)	Horizontal-friendly
Best for	Relations, transactions, integrity	High volume, flexible/varied data, scale
Types	—	Document, key-value, column, graph

ACID = Atomicity, Consistency, Isolation, Durability. Know these letters — they pair naturally with the CQRS/consistency discussion. The JD lists SQL as required and NoSQL as an asset, so anchor on relational and mention where NoSQL fits (e.g. a denormalized CQRS read store).

9 · .NET Stack

Q Likely .NET talking points LIKELY

Note: the JD requires "excellent knowledge of the .NET stack." Be ready to speak to it at least conceptually. Bridge from what you know — the *architecture* concepts (REST, CQRS, DI, layering) transfer directly; the .NET specifics are the vocabulary layer on top.

- **ASP.NET Core** — cross-platform framework for building REST APIs; middleware pipeline; built-in **dependency injection** (ties straight to SOLID's D).
- **Minimal APIs vs. Controllers** — two styles for defining endpoints.
- **Entity Framework Core** — ORM; `DbContext`, migrations, LINQ queries.
- **MediatR** — common library for implementing CQRS/mediator in .NET (send commands/queries to handlers).
- **async/await** — `Task`-based asynchrony throughout the stack; important for scalable I/O.
- **Model binding & validation** — data annotations / FluentValidation.
- **System.Text.Json** — serialization; relevant to the enum-handling discussion above.

Rapid-Fire Cheat Sheet

Prompt	Anchor answer
REST in one line	Stateless, resource-oriented style over HTTP; verbs = operations.
How to version	URI / query / header / media-type; goal = evolve without breaking consumers.
Breaking change	Forces consumers to change code → new major version.
Enum + new value	Non-breaking in requests, potentially breaking in responses.
Tolerant reader	Ignore unknown fields; always a default enum case.

CQRS	Separate read/write models; optimize & scale each independently.
CQRS vs REST	Internal pattern vs external style — REST can sit on CQRS.
Pod vs container	Pod wraps container(s) sharing network/storage/lifecycle; K8s schedules pods.
Idempotent verbs	GET, PUT, DELETE yes; POST no.
Bounded context	DDD boundary of model consistency $\approx$ service boundary.
Conway's Law	System structure mirrors org communication structure.
ACID	Atomicity, Consistency, Isolation, Durability.